

Experiment_7_XOR_Backpropagation_Momentum.m × +	
C:\Users\Afroz\Desktop\NN\EXP 7\Code\Experiment_7_XOR_Backpropagation_Momentum.m	
1	clc;
2	clear;
3	close all;
4	
5	%% XOR Function Using Backpropagation with Momentum
6	
7	% XOR training data
8	X = [0 0;
9	0 1;
10	1 0;
11	1 1];
12	
13	% Target outputs
14	T = [0;
15	1;
16	1;
17	0];
18	
19	%% Network Architecture
20	% 2 Input Neurons -> 2 Hidden Neurons -> 1 Output Neuron
21	
22	inputNeurons = 2;
23	hiddenNeurons = 2;
24	outputNeurons = 1;
25	
26	%% Learning Parameters
27	learningRate = 0.5;
28	momentum = 0.9;
29	maxEpochs = 10000;
30	targetMSE = 0.001;
31	
32	%% Initialize Weights and Biases
33	rng(1);
34	
35	W_ih = randn(inputNeurons, hiddenNeurons) * 0.5;
36	b_h = randn(1, hiddenNeurons) * 0.5;
37	
38	W_ho = randn(hiddenNeurons, outputNeurons) * 0.5;
39	b_o = randn(1, outputNeurons) * 0.5;
40	
41	%% Previous Weight Changes for Momentum
42	dW_ih_prev = zeros(size(W_ih));
43	db_h_prev = zeros(size(b_h));
44	
45	dW_ho_prev = zeros(size(W_ho));
46	db_o_prev = zeros(size(b_o));
47	
48	%% Sigmoid Function
49	sigmoid = @(x) 1 ./ (1 + exp(-x));
50	

51	%% Training
52	MSE_history = zeros(maxEpochs, 1);
53	
54	for epoch = 1:maxEpochs
55	
56	totalError = 0;
57	
58	for i = 1:size(X,1)
59	
60	%% Forward Propagation
61	
62	% Hidden layer
63	net_h = X(i,:) * W_ih + b_h;
64	h = sigmoid(net_h);
65	
66	% Output layer
67	net_o = h * W_ho + b_o;
68	y = sigmoid(net_o);
69	
70	%% Error
71	error = T(i) - y;
72	
73	totalError = totalError + error^2;
74	
75	%% Output Delta
76	delta_o = error * y * (1 - y);
77	
78	%% Hidden Layer Delta
79	delta_h = (h .* (1 - h)) .* (delta_o * W_ho');
80	
81	%% Weight Updates with Momentum
82	
83	% Hidden to Output
84	dW_ho = learningRate * (h' * delta_o) ...
85	+ momentum * dW_ho_prev;
86	
87	db_o = learningRate * delta_o ...
88	+ momentum * db_o_prev;
89	
90	% Input to Hidden
91	dW_ih = learningRate * (X(i,:) * delta_h) ...
92	+ momentum * dW_ih_prev;
93	
94	db_h = learningRate * delta_h ...
95	+ momentum * db_h_prev;
96	

97	%% Update Weights and Biases
98	
99	W_ho = W_ho + dW_ho;
100	b_o = b_o + db_o;
101	
102	W_ih = W_ih + dW_ih;
103	b_h = b_h + db_h;
104	
105	%% Store Previous Updates
106	
107	dW_ho_prev = dW_ho;
108	db_o_prev = db_o;
109	
110	dW_ih_prev = dW_ih;
111	db_h_prev = db_h;
112	
113	end
114	
115	%% Mean Squared Error
116	
117	MSE = totalError / size(X,1);
118	MSE_history(epoch) = MSE;
119	
120	if mod(epoch,100) == 0
121	fprintf('Epoch %d: MSE = %.6f\n', epoch, MSE);
122	end
123	
124	%% Stop if Target MSE is Reached
125	
126	if MSE <= targetMSE
127	
128	fprintf('\nTraining converged at Epoch %d\n', epoch);
129	fprintf('Final MSE = %.6f\n', MSE);
130	
131	break;
132	
133	end
134	
135	end
136	

```

137 %% Testing the Trained Network
138
139 fprintf('\nXOR Classification Results\n');
140 fprintf('-----\n');
141 fprintf('X1\tX2\tTarget\tNetwork Output\tBinary Output\n');
142 fprintf('-----\n');
143
144 for i = 1:size(X,1)
145
146     % Forward propagation
147
148     net_h = X(i,:) * W_ih + b_h;
149     h = sigmoid(net_h);
150
151     net_o = h * W_ho + b_o;
152     y = sigmoid(net_o);
153
154     % Binary output using threshold 0.5
155
156     if y >= 0.5
157         binaryOutput = 1;
158     else
159         binaryOutput = 0;
160     end
161
162     fprintf('%d\t%d\t%d\t%.4f\t\t%d\n', ...
163           X(i,1), X(i,2), T(i), y, binaryOutput);
164
165 end
166
167 %% Plot Training Error
168
169 figure;
170
171 plot(1:epoch, MSE_history(1:epoch), 'LineWidth', 2);
172
173 xlabel('Epoch');
174 ylabel('Mean Squared Error');
175 title('XOR Backpropagation with Momentum');
176
177 grid on;

```

Command Window

Epoch 100: MSE = 0.177939
Epoch 200: MSE = 0.106415
Epoch 300: MSE = 0.003074
Epoch 400: MSE = 0.001442

Training converged at Epoch 482
Final MSE = 0.000997

XOR Classification Results

X1	X2	Target	Network Output	Binary Output
0	0	0	0.0265	0
0	1	1	0.9695	1
1	0	1	0.9708	1
1	1	0	0.0387	0

>>

